instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 7

1

Exercício 4 - Usar outra fórmula

Tarefa

Pelo dia anterior, O produtor 1 vertia as notas e você viu com seu descanso do anterior. O produtor 2 vertia as notas e você viu com um descanso de um.

Escreva uma função que recebe `descanso_do_produtor` e `produtor`, para cada produtor, diz quanto de descanso (a função de descanso da pergunta de descanso por ao), do valor do descanso e do preço a pagar.

Para calcular o preço dos produtos use a fórmula: `preço = 5 - descanso/2`

Solução

Use o programa do exercício 2 e altere a fórmula de cálculo do preço.

Pretende-se manter inalterada a solução do exercício 2, por isso copiou-se e falsificou-se a função `preço_a_pagar` para `preço_a_pagar_nova_formula`.

Programa (ou ficheiro) `exercicio_4_preco_a_pagar_nova_formula.py`

Escreva e execute este programa. Confirme o seu output.

```
# -*- coding: utf-8 -*-
# preço_a_pagar_nova_formula(descanso_do_produtor, produtor)
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
# preço_a_pagar_nova_formula(descanso_do_produtor, produtor) é o preço a pagar
def preço_a_pagar_nova_formula(descanso_do_produtor, produtor):
    preço = 5 - descanso_do_produtor/2
    return preço

# Testes
print("preço_a_pagar_nova_formula(0, 1) =", preço_a_pagar_nova_formula(0, 1))
print("preço_a_pagar_nova_formula(1, 1) =", preço_a_pagar_nova_formula(1, 1))
print("preço_a_pagar_nova_formula(2, 1) =", preço_a_pagar_nova_formula(2, 1))
print("preço_a_pagar_nova_formula(0, 2) =", preço_a_pagar_nova_formula(0, 2))
print("preço_a_pagar_nova_formula(1, 2) =", preço_a_pagar_nova_formula(1, 2))
print("preço_a_pagar_nova_formula(2, 2) =", preço_a_pagar_nova_formula(2, 2))
```

Output

```
preço_a_pagar_nova_formula(0, 1) = 5.0
preço_a_pagar_nova_formula(1, 1) = 4.5
preço_a_pagar_nova_formula(2, 1) = 4.0
preço_a_pagar_nova_formula(0, 2) = 5.0
preço_a_pagar_nova_formula(1, 2) = 4.5
preço_a_pagar_nova_formula(2, 2) = 4.0
```

Inacreditável!!!

- Parece que uma pequenissíssima mudança esta solução inacreditável?
- Pois, mudou a fórmula alterou o valor. Parece a fórmula só veio assim ela. Só a função `preço_a_pagar` é que mudou. Sempre que se quiser alterar o que a função faz, basta alterar a função e fazemos o `preço = preço - descanso/2`.

Observação

- Note-se que uma parte do programa está repetida, essa parte do programa deve ser transformada numa função.
- E isso ser-se-á tão-pronto se se acaso, também com o cálculo do fator de descanso que também se repete. Se descansou, não também, não repete-se-á também.
- Para além disso também se nasceu variáveis em vez de valores fixos. Para evitar repetir valores.

Programa (ou ficheiro) `exercicio_4_preco_a_pagar_nova_formula_v2.py`

Escreva e execute este programa. Confirme o seu output.

```
# preço = 5 - dia
# preço_a_pagar(dia) é o preço a pagar
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
def preço_a_pagar_nova_formula(descanso_do_produtor, produtor):
    preço = 5 - descanso_do_produtor/2
    return preço

# Testes
for descanso_do_produtor in [0, 1, 2]:
    for produtor in [1, 2]:
        print("preço_a_pagar_nova_formula(", descanso_do_produtor, ",", produtor, ") =", preço_a_pagar_nova_formula(descanso_do_produtor, produtor))
```

Output

```
preço_a_pagar_nova_formula( 0 , 1 ) = 5.0
preço_a_pagar_nova_formula( 0 , 2 ) = 5.0
preço_a_pagar_nova_formula( 1 , 1 ) = 4.5
preço_a_pagar_nova_formula( 1 , 2 ) = 4.5
preço_a_pagar_nova_formula( 2 , 1 ) = 4.0
preço_a_pagar_nova_formula( 2 , 2 ) = 4.0
```

Observação

- Note-se que uma parte do programa está repetida, essa parte do programa deve ser transformada numa função.
- E isso ser-se-á tão-pronto se se acaso, também com o cálculo do fator de descanso que também se repete. Se descansou, não também, não repete-se-á também.
- Para além disso também se nasceu variáveis em vez de valores fixos. Para evitar repetir valores.

Programa (ou ficheiro) `exercicio_4_preco_a_pagar_nova_formula_v3.py`

Escreva e execute este programa. Confirme o seu output.

```
# preço = 5 - dia
# preço_a_pagar(dia) é o preço a pagar
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
def preço_a_pagar_nova_formula(descanso_do_produtor, produtor):
    preço = 5 - descanso_do_produtor/2
    return preço

# Testes
for descanso_do_produtor in [0, 1, 2]:
    for produtor in [1, 2]:
        print("preço_a_pagar_nova_formula(", descanso_do_produtor, ",", produtor, ") =", preço_a_pagar_nova_formula(descanso_do_produtor, produtor))
```

Output

```
preço_a_pagar_nova_formula( 0 , 1 ) = 5.0
preço_a_pagar_nova_formula( 0 , 2 ) = 5.0
preço_a_pagar_nova_formula( 1 , 1 ) = 4.5
preço_a_pagar_nova_formula( 1 , 2 ) = 4.5
preço_a_pagar_nova_formula( 2 , 1 ) = 4.0
preço_a_pagar_nova_formula( 2 , 2 ) = 4.0
```

Observação

- Note-se que uma parte do programa está repetida. Por exemplo, na primeira versão o output das várias versões mostrando com o cálculo dos valores (as variáveis versus os valores). Agora temos uma versão do programa bem definida.
  - Uma parte inicial de definição de variáveis. Não se dados que nunca mudam.
  - Uma segunda parte com a definição das funções. Se não se dados que nunca mudam.
  - Uma terceira parte com os valores obtidos.
- Não se dados uma quarta parte com o output dos valores obtidos (`print`).
- Esta versão dispensa-nos de comentários. Porque esta estrutura contribui para automatizar a clareza e consequentemente a legibilidade do programa.

Funções

- Uma função é uma parte de uma programa. Essa parte do programa é reutilizável. Pode ser usada em várias sítios.
- Uma função tem um nome, uma sequência de argumentos, e um corpo.
- O nome da função é a maneira de a chamar. Podemos chamar a função `preço` três sítios.
- O professor cria-se nomes próprios funções? Para fazermos o que são queremos.
- Uma função recebe valores.
- O que tem que ao linguagem de programação são não professor. Porque podemos nomeadamente nomeadamente nomeadamente nomeadamente, que são coisinhas. A classe nos cria a função `preço_a_pagar`.

Definição de Funções

- Para definir uma função usa-se a instrução `def`. Isto se letras iniciais de palavra definição.
- Uma função tem um nome, uma sequência de argumentos, e um corpo.
- O nome da função é a maneira de a chamar. Podemos chamar a função `preço` três sítios.
- O professor cria-se nomes próprios funções? Para fazermos o que são queremos.
- Uma função recebe valores.
- O que tem que ao linguagem de programação são não professor. Porque podemos nomeadamente nomeadamente nomeadamente nomeadamente, que são coisinhas. A classe nos cria a função `preço_a_pagar`.

Exemplo (ao ficheiro `definicao_da_funcao.py`)

Escreva e execute este programa. Qual é o output?

```
# uma nova função chamada preço_a_pagar
# a função recebe dois argumentos, descanso e produtor
# a função devolve o valor do preço, que é o cálculo da função
# se produtor é igual a 1 então faz o cálculo da função
# se produtor é igual a 2 então faz o cálculo da função
# se produtor não é igual a 1 nem a 2 então faz o cálculo da função
def preço_a_pagar(descanso, produtor):
    if produtor == 1:
        return 5 - descanso/2
    if produtor == 2:
        return 5 - descanso/2
    return 5 - descanso/2

print("Hello World")
```

Output

```
Hello World
```

Observação

- O output é só `Hello World` mas por a função ficou definida mas não foi chamada. Se não se chamar a função é como se não existisse. Se não é executado **ao** que diz respeito da função.

Chamar funções

- Para executar uma função chama-se a função. Para chamar a função usa-se o nome da função seguido de parentesis curvos. E fornecem-se valores valores quando o número de argumentos da função, separados por vírgulas. Estes valores passam-se valores.

Exemplo (ao ficheiro `chamar_funcao.py`)

Escreva e execute este programa. Qual é o output?

```
def preço_a_pagar(dia, descanso):
    preço = descanso * 5 - dia
    return preço

print("descanso = 0 e produtor = 1")
print("o valor do descanso é", 0)
valor_do_descanso = 0
print("o valor do preço a pagar é", preço_a_pagar(0, valor_do_descanso))
print("descanso = 0 e produtor = 2")
print("o valor do descanso é", 0)
valor_do_descanso = 0
print("o valor do preço a pagar é", preço_a_pagar(0, valor_do_descanso))

print("Hello World")

print("descanso = 0 e produtor = 3")
print("o valor do descanso é", 0)
valor_do_descanso = 0
print("o valor do preço a pagar é", preço_a_pagar(0, valor_do_descanso))

print("descanso = 1 e produtor = 1")
print("o valor do descanso é", 1)
valor_do_descanso = 1
print("o valor do preço a pagar é", preço_a_pagar(1, valor_do_descanso))

print("descanso = 1 e produtor = 2")
print("o valor do descanso é", 1)
valor_do_descanso = 1
print("o valor do preço a pagar é", preço_a_pagar(1, valor_do_descanso))

print("descanso = 1 e produtor = 3")
print("o valor do descanso é", 1)
valor_do_descanso = 1
print("o valor do preço a pagar é", preço_a_pagar(1, valor_do_descanso))

print("descanso = 2 e produtor = 1")
print("o valor do descanso é", 2)
valor_do_descanso = 2
print("o valor do preço a pagar é", preço_a_pagar(2, valor_do_descanso))

print("descanso = 2 e produtor = 2")
print("o valor do descanso é", 2)
valor_do_descanso = 2
print("o valor do preço a pagar é", preço_a_pagar(2, valor_do_descanso))

print("descanso = 2 e produtor = 3")
print("o valor do descanso é", 2)
valor_do_descanso = 2
print("o valor do preço a pagar é", preço_a_pagar(2, valor_do_descanso))
```

Output

```
descanso = 0 e produtor = 1
o valor do descanso é 0
o valor do preço a pagar é 0
descanso = 0 e produtor = 2
o valor do descanso é 0
o valor do preço a pagar é 0
Hello World
descanso = 0 e produtor = 3
o valor do descanso é 0
o valor do preço a pagar é 0
descanso = 1 e produtor = 1
o valor do descanso é 1
o valor do preço a pagar é 4
descanso = 1 e produtor = 2
o valor do descanso é 1
o valor do preço a pagar é 4
descanso = 1 e produtor = 3
o valor do descanso é 1
o valor do preço a pagar é 4
descanso = 2 e produtor = 1
o valor do descanso é 2
o valor do preço a pagar é 3
descanso = 2 e produtor = 2
o valor do descanso é 2
o valor do preço a pagar é 3
descanso = 2 e produtor = 3
o valor do descanso é 2
o valor do preço a pagar é 3
```

Mais sobre chamar funções

- Uma função só pode ser chamada depois de ter sido definida.

Exemplo (ao ficheiro `chamar_funcao_nao_definida.py`)

Escreva e execute este programa. Qual é o output?

```
preço = 5 - dia
# uma função só pode ser usada depois de ter sido definida
def preço_a_pagar(dia, descanso):
    preço = 5 - dia
    return preço

# uma nova função chamada preço_a_pagar
# a função recebe dois argumentos, descanso e produtor
# a função devolve o valor do preço, que é o cálculo da função
# se produtor é igual a 1 então faz o cálculo da função
# se produtor é igual a 2 então faz o cálculo da função
# se produtor não é igual a 1 nem a 2 então faz o cálculo da função
print("a executar a função preço_a_pagar ...")
def preço_a_pagar(dia, descanso):
    if produtor == 1:
        return 5 - dia/2
    if produtor == 2:
        return 5 - dia/2
    return 5 - dia/2

print("a executar a função preço_a_pagar ... feita")

print("Hello World")
```

Output

```
Traceback (most recent call last):
  File "/home/runner/Exercicio-4-Usar-outra-formula/chamar_funcao_nao_definida.py", line 5, in <module>
    preço = preço_a_pagar(dia, descanso)
NameError: name 'preço_a_pagar' is not defined
```

Argumentos

- Uma função pode receber valores para nomes quando são executadas. Para isso nomes os argumentos na definição das funções. Os argumentos são os nomes que aparecem dentro dos parentesis curvos.
- Os argumentos são funções são **obrigatórios**.
- Isto é, ao uma função é definida, por exemplo, com 2 argumentos, quando é chamada tem que ser com 2 argumentos.
- Uma função pode não ter argumentos. Mas a chamada à função tem que ter os parentesis curvos mesmo vazio. Mais sobre isso mais tarde.

Exemplo (ao ficheiro `argumentos_funcao.py`)

Escreva e execute este programa. Qual é o output?

```
# preço = 5 - dia
# preço_a_pagar(dia) é o preço a pagar
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
# preço_a_pagar(dia, descanso_do_produtor, produtor) é o preço a pagar
def preço_a_pagar(dia, descanso_do_produtor, produtor):
    preço = 5 - dia
    return preço

# Testes
print("preço_a_pagar(0, 0, 1) =", preço_a_pagar(0, 0, 1))
print("preço_a_pagar(1, 0, 1) =", preço_a_pagar(1, 0, 1))
print("preço_a_pagar(2, 0, 1) =", preço_a_pagar(2, 0, 1))
print("preço_a_pagar(0, 0, 2) =", preço_a_pagar(0, 0, 2))
print("preço_a_pagar(1, 0, 2) =", preço_a_pagar(1, 0, 2))
print("preço_a_pagar(2, 0, 2) =", preço_a_pagar(2, 0, 2))
```

Output

```
preço_a_pagar(0, 0, 1) = 5
preço_a_pagar(1, 0, 1) = 4
preço_a_pagar(2, 0, 1) = 3
preço_a_pagar(0, 0, 2) = 5
preço_a_pagar(1, 0, 2) = 4
preço_a_pagar(2, 0, 2) = 3
```

Observação

- Note-se que o número de argumentos é 3 mas o número de argumentos é 2. Isto é, o número de argumentos é 2 mas o número de argumentos é 3.
- Note-se que o número de argumentos é 3 mas o número de argumentos é 2. Isto é, o número de argumentos é 2 mas o número de argumentos é 3.

Exemplo (ao ficheiro `argumentos_funcao_em_falta.py`)

Escreva e execute este programa. Qual é o output?

```
# preço = 5 - dia
# preço_a_pagar(dia) é o preço a pagar
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
# preço_a_pagar(dia, descanso_do_produtor, produtor) é o preço a pagar
def preço_a_pagar(dia, descanso_do_produtor, produtor):
    preço = 5 - dia
    return preço

# Testes
print("preço_a_pagar(0, 0, 1) =", preço_a_pagar(0, 0, 1))
print("preço_a_pagar(1, 0, 1) =", preço_a_pagar(1, 0, 1))
print("preço_a_pagar(2, 0, 1) =", preço_a_pagar(2, 0, 1))
print("preço_a_pagar(0, 0, 2) =", preço_a_pagar(0, 0, 2))
print("preço_a_pagar(1, 0, 2) =", preço_a_pagar(1, 0, 2))
print("preço_a_pagar(2, 0, 2) =", preço_a_pagar(2, 0, 2))
```

Output

```
preço_a_pagar(0, 0, 1) = 5
preço_a_pagar(1, 0, 1) = 4
preço_a_pagar(2, 0, 1) = 3
preço_a_pagar(0, 0, 2) = 5
preço_a_pagar(1, 0, 2) = 4
preço_a_pagar(2, 0, 2) = 3
```

Retorno

- Uma função pode retornar um valor. É útil para que se possa armazenar numa variável o valor retornado pela função. Para tal usa-se a instrução `return` seguido do valor a retornar.
- Uma função só pode retornar o exercício a função. Nada mais é executado ao corpo da função depois ser ser executado o `return`.
- A instrução `return` é obrigatória. Mas uma função retorna sempre um valor. Quando não se usa a instrução `return` a função retorna `None`.

Exemplo (ao ficheiro `retorno_funcao.py`)

Escreva e execute este programa. Qual é o output?

```
# uma função que retorna o preço a pagar
def preço_a_pagar(dia, descanso, produtor):
    preço = 5 - dia
    print("preço = descanso*5 - dia é o mesmo que preço = 5 - dia")
    print("preço = descanso*5 - dia é o mesmo que preço = 5 - dia")
    return preço
    print("esta linha não é executada")

# a primeira print nunca é executada porque está depois do return
# a segunda print nunca é executada porque está depois do return
# a terceira print nunca é executada porque está depois do return

print("Obrigado pela sua visita!")
print("Obrigado pela sua visita!")
print("Obrigado pela sua visita!")

agradeço_a_visita()
agradeço_a_visita()
agradeço_a_visita()
```

Output

```
Obrigado pela sua visita!
Obrigado pela sua visita!
Obrigado pela sua visita!
```

Contexto

Variáveis locais

- Os argumentos da lista de argumentos de definição da função são variáveis locais à função. Isto quer dizer que não estão acessíveis fora da função e deixam de existir depois a função ser executada.
- As variáveis inicializadas dentro de uma função também são locais à função (também não estão acessíveis fora da função e deixam de existir).

Variáveis globais

- Variáveis inicializadas fora das funções são designadas de variáveis globais. As funções podem usar as variáveis globais. As variáveis globais estão acessíveis de funções.

Exemplo (ao ficheiro `variaveis_locais.py`)

Escreva e execute este programa. Qual é o seu output?

```
# uma função que retorna o preço a pagar
def preço_a_pagar(dia, descanso, produtor):
    preço = 5 - dia
    print("preço = descanso*5 - dia é o mesmo que preço = 5 - dia")
    print("preço = descanso*5 - dia é o mesmo que preço = 5 - dia")
    return preço
    print("esta linha não é executada")

# a primeira print nunca é executada porque está depois do return
# a segunda print nunca é executada porque está depois do return
# a terceira print nunca é executada porque está depois do return

print("Obrigado pela sua visita!")
print("Obrigado pela sua visita!")
print("Obrigado pela sua visita!")

agradeço_a_visita()
agradeço_a_visita()
agradeço_a_visita()
```

Output

```
Obrigado pela sua visita!
Obrigado pela sua visita!
Obrigado pela sua visita!
```

Para fazer

Considere o programa Python. E que se segue:

```
# -*- coding: utf-8 -*-
# preço = 5 - dia
# preço_a_pagar(dia) é o preço a pagar
# descanso_do_produtor é o descanso do produtor
# produtor é o produtor
# preço_a_pagar(dia, descanso_do_produtor, produtor) é o preço a pagar
def preço_a_pagar(dia, descanso_do_produtor, produtor):
    preço = 5 - dia
    return preço

# Testes
print("preço_a_pagar(0, 0, 1) =", preço_a_pagar(0, 0, 1))
print("preço_a_pagar(1, 0, 1) =", preço_a_pagar(1, 0, 1))
print("preço_a_pagar(2, 0, 1) =", preço_a_pagar(2, 0, 1))
print("preço_a_pagar(0, 0, 2) =", preço_a_pagar(0, 0, 2))
print("preço_a_pagar(1, 0, 2) =", preço_a_pagar(1, 0, 2))
print("preço_a_pagar(2, 0, 2) =", preço_a_pagar(2, 0, 2))
```

Confirme que o *output* do programa é o que se segue.

```
preço_a_pagar(0, 0, 1) = 5
preço_a_pagar(1, 0, 1) = 4
preço_a_pagar(2, 0, 1) = 3
preço_a_pagar(0, 0, 2) = 5
preço_a_pagar(1, 0, 2) = 4
preço_a_pagar(2, 0, 2) = 3
```

Altere o programa o código que faz pretenda indicar se as afirmações que se seguem são verdadeiras ou falsas.

```
preço_a_pagar(0, 0, 1) = 5
preço_a_pagar(1, 0, 1) = 4
preço_a_pagar(2, 0, 1) = 3
preço_a_pagar(0, 0, 2) = 5
preço_a_pagar(1, 0, 2) = 4
preço_a_pagar(2, 0, 2) = 3
```

1.1

Considere o programa:

```
def c(p, i):
    print(p)
    print(i)
p = 1
i = 2
c(i, p)
```

A execução deste programa dá erro.

1.2

Considere o programa:

```
def j():
    m = 1
s = j()
print(s)
```

O *output* que o prograna gera é:

```
None
```

1.3

Considere o programa:

```
def q(v):
    print(v)
print('w')
```

O *output* que o prograna gera é:

```
w
```

1.4

Considere o programa:

```
def d(b):
    print(b)
print('f')
d('b')
```

O *output* que o prograna gera é:

```
f
b
```

1.5

```
def k(u, e):
    h = u + e
    return h
n = k(4)
print(n)
```

A execução deste programa não dá erro.